Array 29 (2026) 100681

Contents lists available at ScienceDirect

Array

journal homepage: www.elsevier.com/locate/array

An automated multi-scale and multi-contextual MobileNetv3 for malware detection based on IoT

Sidra Javed [a], Guowei Wu [a], Hamza Javed [b], Osama A. Khashan [c], Haseeb Hassan [d], Anwar Ghani [e,*]

[a] School of Software Technology, Dalian University of Technology, Dalian, 116024, Liaoning, China
[b] School of Computer Science and Engineering, Central South University, Changsha, 410017, Hunan, China
[c] Research and Innovation Centers, Rabdan Academy, 114646, United Arab Emirates
[d] School of Medicine, The Chinese University of Hong Kong, Shenzhen, 518172, China
[e] Department of Computer Science, School of Engineering and Digital Sciences, Nazarbayev University, Astana, 010000, Kazakhstan

ARTICLE INFO

Dataset link: https://www.kaggle.com/datasets/manmandes/malimg, https://web.cs.hacettepe.edu.tr/~selman/malevis/

Keywords:
Malware detection
Deep learning
Multi-scale features
MobileNetv3
IoT

ABSTRACT

Malware detection is a crucial aspect of cybersecurity, aimed at identifying and mitigating malicious software that poses threats to systems and networks. Traditional malware detection methods face challenges in terms of both detection accuracy and computational cost, as deep learning models can be resource-intensive and difficult to deploy in real-time environments. This paper introduces the novel MSMC-MobileNet (Multi-Scale and Multi-Contextual MobileNet) malware detection and classification model, designed to address the challenges of accuracy and computational cost. First, MobileNetv3 is used to extract features from the dataset. To enhance feature extraction, the SE (Squeeze-and-Excitation) module is integrated, focusing on the region of interest using an attention mechanism. The multiscale and multicontextual features are extracted using the ASPP (Atrous Spatial Pyramid Pooling) and FPP (Feature Pyramid Pooling) modules. Channel-wise pruning is applied to the ASPP and FPP modules, reducing computational cost. The model is evaluated on the publicly available Malimg and MaleVis datasets. The proposed MSMC-MobileNet model achieves impressive performance with 92.37% accuracy, 96.54% precision, 95.84% recall, 95.47% F1 score, and 98.59% AUC on the Malimg dataset. On the MaleVis dataset, the model yields 95.08% accuracy, 98.33% precision, 97.9% recall, 98.15% F1 score, and 96.98% AUC. When both datasets are combined, the MSMC-MobileNet achieves 98.79% accuracy, 99.84% precision, 99.73% recall, 99.89% F1 score, and 1.00 AUC. Despite its high accuracy, the model remains computationally efficient, outperforming state-of-the-art methods in both detection performance and computational cost.

1. Introduction

Malware, short for malicious software, refers to any software designed to disrupt, damage, or gain unauthorized access to computer systems. Malware is any software intentionally designed to harm or exploit a computer system, network, or device. It includes various types such as viruses, worms, trojans, ransomware, and spyware [1]. Malware typically aims to disrupt system operations, steal sensitive data, or grant attackers unauthorized access. Its behavior can range from data corruption to surveillance and remote control of infected systems. As cyber threats have evolved, the detection and mitigation of malware have become critical to ensuring the security of digital systems and data [2]. Traditional malware detection methods, such as signature-based detection [3], involve identifying known patterns or signatures of malicious code. However, the increasing sophistication of malware, including polymorphic and zero-day variants [4], has highlighted the need for more advanced detection techniques that can adapt to new and evolving threats. This advanced technique involves using byteplot images, where the binary content of a file is visualized as an image, and patterns within these images are analyzed to detect malware [5]. Byteplot-based malware detection offers the advantage of potentially identifying malicious code through visual patterns, which can be less susceptible to obfuscation techniques that thwart traditional detection methods. However, this approach also presents challenges, such as the need for extensive and diverse datasets to train models effectively, the difficulty of distinguishing between benign and malicious patterns in

* Corresponding author.
E-mail address: anwar.ghani@nu.edu.kz (A. Ghani).

https://doi.org/10.1016/j.array.2026.100681
Received 11 May 2025; Received in revised form 12 January 2026; Accepted 20 January 2026
Available online 20 January 2026

similar-looking byteplots, and the computational cost associated with processing and analyzing high-dimensional image data.

The primary challenges in deep learning malware detection include addressing the increasing sophistication of evolving malware types, such as polymorphic and zero-day variants, which necessitate adaptable detection methods [6]. Effective analysis of byteplot images, used to visualize binary malware content, requires extensive and diverse datasets, which are often unbalanced and limited [7]. Distinguishing between benign and malicious patterns in visually similar byteplots further complicates detection [8]. Computational costs, particularly in processing high-dimensional image data, pose a challenge for real-time and resource-constrained deployments [9]. Robust feature extraction methods are needed to capture multiscale and multicontextual information while minimizing overfitting [10]. Balancing detection accuracy with processing efficiency and scaling models to handle diverse datasets remains an ongoing research concern in this domain.

Deep learning approaches have shown promise in addressing these challenges, particularly through the use of convolutional neural networks (CNNs) and other architectures that excel in image recognition tasks [11]. Using deep learning, it is possible to automatically learn features from byteplot images that are indicative of malware, without relying on hand-crafted rules or signatures. This enables the detection of previously unseen malware and the ability to generalize across various types of threats. Furthermore, techniques such as transfer learning [12] and ensemble methods [13] can enhance the performance of deep learning models, making them more robust in detecting a wide range of malware while reducing false positives and computational costs.

The approach presented in this research targets a broad spectrum of malware, including Adware, Trojans, Viruses, Worms, and Backdoors. It focuses on detecting malware based on binary content visualized as byteplot images. The model is not limited to any single malware family but instead aims to generalize across a wide variety of malicious software. In this research, a novel MSMC-MobileNet is used for automated malware classification using byteplot images. The standard architecture of MobileNetv3 [14] is modified by adding Atrous Spatial Pyramid Pooling (ASPP) [15] and Feature Pyramid Pooling (FPP) [16] to extract multi-scale and multi-contextual features. After that, the MobileNetv3 output is passed to additional layers for further processing, such as ASPP, FPP, and Squeeze-and-Excitation (SE) blocks [17]. The experiments are performed using preprocessing, data augmentation, and dropout regularization. The proposed model is evaluated using MaleVis and Malimg datasets. Based on the results, the proposed model outperforms state-of-the-art methods. The main contributions of the proposed model are as follows:

- To introduce MSMC-MobileNet, a modified version of MobileNetv3, designed for the automated classification of malware using byteplot images, incorporating modules to enhance feature extraction capabilities.
- The architecture includes ASPP and FPP modules to capture multi-scale and multi-contextual features, significantly improving the model's ability to classify diverse malware types.
- The use of SE modules enables channel-wise feature extraction, allowing the model to focus on important information while suppressing less relevant data, thereby improving classification accuracy and robustness.
- The model leverages the lightweight MobileNetv3 architecture, ensuring low computational cost, and making it suitable for real-time deployment on resource-constrained devices while maintaining high accuracy.
- The proposed model achieves state-of-the-art results on the Malimg and MaleVis datasets, with accuracy, precision, recall, F1 score, and AUC outperforming existing methods, demonstrating its effectiveness in malware detection and classification.

The remainder of the article is structured as follows: Section 2 reviews existing methods for malware detection in IoT environments. Section 3 explains the proposed MSMC-MobileNet model, focusing on its architecture and feature extraction modules (SE, ASPP, FPP). Section 4 outlines the experimental setup, including datasets, evaluation metrics, and results, which compare the proposed model with existing approaches. Section 5 concludes with a summary of the paper's contributions and future research directions.

2. Related work

Malware detection has become a critical area of research in cybersecurity due to the increasing sophistication and frequency of malicious attacks [18,19]. Traditional signature-based detection methods, while compelling against known threats, struggle to keep pace with the evolving nature of malware. As a result, researchers have increasingly turned to machine learning and deep learning techniques, which offer the ability to detect previously unknown malware by learning patterns and behaviors from large datasets [20]. Techniques such as static and dynamic analysis [21] and hybrid models [22], which combine multiple detection approaches, have shown promise in improving accuracy and reducing false positives.

Jian et al. [23] presented a novel malware detection and family classification framework called SERLA, which utilizes visualization technology and deep neural networks, specifically combining SEResNet50, Bi-LSTM, and attention mechanisms. SERLA demonstrates superior performance in accuracy, precision, recall, and F1-score compared to several state-of-the-art models using the complete BIG 2015 dataset. The framework emphasizes the importance of feature extraction and the advantages of using RGB images over grayscale images. Despite longer training times, SERLA's prediction speed is comparable to other models. The paper has some limitations, including potential misclassification due to similarities between benign software and malware and variations in compilation configurations.

Islam et al. [24] presented "Jadeite", a novel approach for detecting Java malware using deep learning and static analysis techniques. It focuses on transforming Java bytecode into grayscale images by analyzing the Inter Procedural Control Flow Graph (ICFG) and extracting relevant features. The method employs a Convolutional Neural Network (CNN) for classification, achieving 98.4% accuracy on a dataset of Java applications. The study highlights advancements in malware detection, particularly in handling obfuscation techniques used by malware developers. Jadeite struggles with active code obfuscation, which can hinder its detection capabilities. The tool requires further testing with larger datasets to validate its effectiveness. Challenges remain in selecting appropriate features for malware detection, as traditional features may not adequately capture complex malware behaviors.

A novel Bi-model architecture for malware detection was presented by Furqan et al. [25] that combines features from pre-trained models VGG16 and ResNet-50, achieving high accuracy in classifying malware types. The proposed approach outperforms traditional machine learning models (SVC, RF, LR) and deep learning models (CNN, InceptionV3, EfficientNetB0) on the Malimg dataset, achieving 100% accuracy with Bi-SVC, Bi-RF, and Bi-LR. The study emphasizes the effectiveness of combining transfer learning with traditional machine learning techniques for improved malware classification, supported by extensive experiments and statistical validation. The study mentions plans to expand the dataset, suggesting the current dataset may be limited in size and potentially affecting the generalizability of the results. Some classes, particularly 20 and 21, showed lower performance metrics, suggesting classification challenges that may require further analysis and improvement.

A multimodal malware classification framework was proposed by Sadia et al. [26] using an ensemble deep neural network. The framework integrates numerical and imagery data using a late-fusion approach to improve detection accuracy. It employs techniques like

Neighborhood Component Analysis (NCA) for feature selection, Synthetic Minority Over-sampling Technique (SMOTE) for data balancing, and Random Under-Sampling with Boosting (RUSBoost) to handle class imbalance. The experimental results demonstrate that the proposed model outperforms standalone numerical or image-based methods, achieving 95.36% accuracy in multimodal data fusion. This performance improvement highlights the robustness of combining both data types for malware detection, effectively identifying a wide range of malware variants. The study emphasizes the advantages of late fusion, mitigating bias and enhancing classification capabilities, and lays the foundation for advanced multimodal cybersecurity systems.

Another novel approach, called MIRACLE (Malware Image Recognition and Classification by Layered Extraction), was proposed by Alam et al. [27] to improve malware detection using deep learning. MIRACLE leverages a spatial convolutional neural network (Sp-CNN) to effectively analyze malware images, identifying and classifying them based on spatial information extracted from binary representations. The system converts malware binary files into grayscale images and applies augmentation techniques to address class imbalance. The proposed method outperforms traditional machine learning and deep learning techniques, achieving high accuracy rates of 99.87% on the MalImg dataset, 99.81% on the Microsoft-Big dataset, and 99.22% on the Malevis dataset. The MIRACLE demonstrates superior performance in comparison to other state-of-the-art models, providing a lightweight and efficient solution for malware classification with minimal computational overhead.

A lightweight malware detection and classification system, MALITE [28], was proposed for resource-constrained devices like mobile phones and IoT devices. MALITE leverages two efficient methods: MALITE-HRF, which uses histogram-based feature extraction combined with a random forest classifier, and MALITE-MN, a lightweight neural network model utilizing residual bottleneck layers. These methods convert malware binaries into images, reducing computational overhead and memory usage. Experimental results on seven open-source datasets show that MALITE models significantly outperform existing approaches in terms of efficiency, requiring far fewer resources while achieving competitive or superior performance in malware classification. The study also demonstrates MALITE's effectiveness in detecting zero-day attacks and its suitability for deployment on low-power, low-memory devices, making it a viable solution for real-time, embedded malware detection. The results suggest that MALITE can balance performance with computational constraints, offering a scalable approach to cybersecurity for constrained environments.

Mohammad et al. [29] introduced a novel approach for malware detection by visualizing executable binaries as grayscale images in the frequency domain using the Discrete Cosine Transform (DCT). It presents a dataset named MaleX, which includes nearly 1 million malware and benign Windows executable samples, and utilizes CNNs for classification, achieving a binary classification accuracy of 96%. The study compares shallow neural networks with deeper architectures like ResNet and proposes a joint feature metric to improve classification accuracy. This method addresses the limitations of traditional static and dynamic analysis techniques, providing a scalable and efficient solution for malware detection.

Jamal et al. [30] discussed the application of deep learning techniques, specifically convolutional neural networks (CNNs) and deep neural networks (DNNs), for malware detection through image analysis. It highlights the challenges posed by the increasing complexity and volume of malware, which traditional antivirus software struggles to address. The authors propose that deep learning methods can enhance malware detection by analyzing visual representations of malware code. The study evaluates various ResNet models, identifying ResNet 152 as the most effective with an accuracy of 93.5%, precision of 93.4%, recall of 95%, and an F1 score of 94.1%. The paper emphasizes the importance of optimizing parameters such as epochs, training loss, and validation loss to improve model performance. It suggests that further research is needed to balance detection accuracy and processing time. The paper does not provide specific limitations regarding the dataset used, the generalizability of the findings, or potential biases in the model training process. It does not address the implications of longer running times for practical applications of the ResNet models in real-world scenarios. Another approach [31] presented a lightweight machine learning framework for real-time IoT malware detection that minimizes computational demands. The proposed framework utilizes an innovative feature extraction technique, Matrix Block Mean Downsampling (MBMD), and incorporates several machine learning algorithms. Experiments conducted using the BODMAS dataset demonstrate the effectiveness of the approach, achieving a better F1-score in detecting IoT malware.

Table 1 showcases a range of state-of-the-art malware detection models, each utilizing distinct features and methodologies to address the evolving complexities of malware. These models leverage advanced techniques, including feature fusion, image-based analysis, and hardware-software integration, to enhance detection performance. By combining multiple input features, they achieve high accuracy and robustness against standard evasion techniques such as obfuscation and polymorphism. The D-CNN Hybrid Mechanism [32] focuses on integrating permission features and API call graphs using Deep Convolutional Neural Networks (DNNs). Enhanced with Ant Colony Optimization and Min-Max Normalization, this method effectively reduces overfitting and handles high-dimensional data, achieving 96.80% accuracy. On the other hand, the MFFCL (Multi-Feature Fusion with Contrastive Learning) [33] employs supervised contrastive learning to mine high-dimensional features, making it particularly effective in resisting code obfuscation, with an impressive precision rate of 97.25%.

The Hybrid Attention Network [34] takes a step further by combining binary and opcode features using cross-attention and triangular attention mechanisms. This model not only excels in capturing local and global relationships within the data but also achieves a near-perfect accuracy of 99.54%, showcasing its effectiveness across datasets of varying sizes. Similarly, the SMASH (Multi-Feature Ensemble Learning) [35] method combines software and hardware features, such as API sequences and memory dumps, to describe malware behavior from multiple dimensions. It is particularly strong in combating evasion techniques, although it faces challenges when hardware-targeted attacks are employed.

Visual approaches such as the Multistage CNN Model [36] and IFMD (Image Fusion for Malware Detection) [37] underscore the growing importance of image-based malware detection. The Multistage CNN Model converts binary files into grayscale images for detection and classification, achieving a commendable 95% accuracy. It is simple yet effective in detecting polymorphic malware, though it may struggle with highly mutated samples. Meanwhile, IFMD enhances this approach by generating RGB images from malware files and applying image fusion techniques to achieve richer feature extraction. With an accuracy of 98.08%, it demonstrates robustness against obfuscation and polymorphism but at the cost of high computational demands.

Existing methods for malware detection face several significant challenges. Traditional signature-based techniques are limited in their effectiveness against zero-day and polymorphic malware, as they rely on predefined patterns that fail to generalize. Static analysis methods struggle with obfuscation and the dynamic nature of modern malware, often requiring computationally expensive feature extraction. Dynamic analysis, while capturing real-time behaviors, is hindered by scalability issues and resource-intensive execution environments. In addition, many machine learning and deep learning approaches incur high computational costs, depend on large, balanced datasets, and exhibit limited robustness to highly mutated malware variants. Common drawbacks include overfitting due to small datasets, misclassification arising from similar visual patterns, and high training complexity in hybrid models, underscoring the need for more efficient and adaptable detection methods.

Table 1
Comparison of malware detection models.

Model	Features	Accuracy (%)	Advantages	Disadvantages
SERLA [23]	Combines SEResNet50, Bi-LSTM, and attention mechanisms using RGB image-based features extracted from malware binaries	99.02	High-dimensional feature extraction, robust against obfuscation, enhanced representation via attention mechanisms	High computational cost and heavy reliance on data augmentation for model generalization
Jadeite [24]	Constructs grayscale images from ICFG and API call sequences for structural representation	98.4	Effective in capturing program behavior through static analysis, resilient to obfuscation techniques	Struggles with dynamic code execution and reflective programming constructs
Bi-Model Transfer Learning [25]	Leverages hybrid image features from pre-trained ResNet-50 and VGG-16 models in a stacked architecture	100	Achieves perfect classification accuracy, highly robust and scalable using transfer learning	Training is resource-intensive and heavily depends on quality of pre-trained models
Bigram-DCT CNN [29]	Converts bigram frequency counts into frequency-domain images using DCT	96	Efficient feature extraction and fast training using frequency-based representations	Less effective for malware with irregular or low-frequency code structures
ResNet for Malware Image Detection [30]	Encodes malware behavioral patterns as images for detection using a general ResNet architecture	–	Adaptable to evolving malware families, leverages deep CNN for general-purpose detection	Requires extensive preprocessing and curated datasets to maintain performance
D-CNN Hybrid Mechanism [32]	Uses permission sets, API call graphs, Ant Colony Optimization, and normalization for malware representation	96.8	Handles high-dimensional inputs well, reduces overfitting by combining CNN and DNN architectures	Demands significant computational resources during feature engineering and model training
MFFCL [33]	Applies supervised contrastive learning on fused static and dynamic features, including API calls and embeddings	97.25	High model stability and resistance to obfuscation, effective fusion of multiple behavioral perspectives	Increased training complexity due to contrastive learning and multi-feature representation
Hybrid Attention Network [34]	Integrates binary and opcode features using triangular and cross-attention mechanisms for enhanced learning	99.54	Performs well across diverse datasets, visual attention enhances focus on relevant code segments	Depends on aligned and high-quality feature datasets for optimal learning
SMASH [35]	Combines API sequences, hardware performance counters, and memory dump features in an ensemble model	–	Leverages both hardware and software-level indicators, strong resilience to evasion attacks	Susceptible to targeted attacks on hardware features, requires secure environment setup
Multistage CNN Model [36]	Converts binary malware into grayscale images and uses multistage classification layers for detection	95	Simple yet effective for polymorphic malware, resistant to common obfuscation techniques	Dependent on visual similarity; struggles with highly mutated or non-visual patterns
IFMD [37]	Uses image fusion from RGB malware images with AlexNet-based CNN for classification	98.08	Rich feature extraction via image fusion, handles polymorphic and obfuscated malware effectively	Computationally heavy due to image generation and fusion operations

3. Methodology

This Section explains the methodology of the MSMC-MobileNet model, which modifies the MobileNetv3 architecture by adding SE, ASPP, and FPP modules to enhance feature extraction. The SE module improves channel-wise feature calibration, while ASPP and FPP capture multi-scale and multi-contextual information for better malware classification. Channel-wise pruning is used to reduce computational cost. The dataset is pre-processed by resizing the images to 224 × 224 for consistency.

3.1. Proposed malware detection model

The proposed deep learning Multi-Scale and Multi-Contextual MSMC-MobileNet methodology for malware detection follows a structured approach, as illustrated in Fig. 1. First, MobileNetv3 is used for feature extraction from malware byteplot images, followed by a convolutional layer. To enhance feature representation, the Squeeze-and-Excitation (SE) module is added, which focuses on essential channel features while reducing computational costs. After another convolutional layer, an Atrous Spatial Pyramid Pooling (ASPP) module is integrated to capture multi-scale contextual features, and a Feature Pyramid Pooling (FPP) module is incorporated to aggregate hierarchical features. To optimize efficiency, both ASPP and FPP undergo channel-wise pruning to eliminate redundant features and reduce computational complexity. The model is trained using the Malimg and MaleVis datasets, employing preprocessing, data augmentation, and dropout regularization to enhance robustness and prevent overfitting. The MSMC-MobileNet model achieves state-of-the-art results, demonstrating high accuracy, precision, recall, F1 score, and AUC, while maintaining computational efficiency for real-time deployment on resource-constrained devices.

To address limitations in feature extraction, the MSMC-MobileNet model integrates an SE module designed explicitly for channel-wise feature recalibration. By applying global average pooling followed by fully connected layers, it assigns importance weights to different channels, emphasizing critical features while suppressing less relevant ones. This enhances the model's ability to identify malware-specific patterns in byteplot images. The ASPP employs parallel atrous (dilated) convolutions with varying rates to capture multi-scale contextual information. This module enables the model to effectively extract features at different resolutions, which is essential for detecting malware patterns that vary in size and scale. The FPP aggregates features from multiple scales and resolutions, providing a comprehensive representation of the input data. This approach enables the model to detect hierarchical patterns, which are often crucial in distinguishing between malware variants.

The MSMC-MobileNet model is designed to detect a variety of malicious behaviors through byteplot image analysis. By analyzing patterns such as unusual byte sequences and changes in file structures, the model can identify these behaviors in malware. The approach captures both local and global image features, multiscale details, hierarchical patterns, and channel-wise dependencies, leveraging modules like SE, ASPP, and FPP. These features help the model focus on both fine-grained and broader contextual patterns, which are essential for detecting malware-specific behaviors like encryption, self-replication,

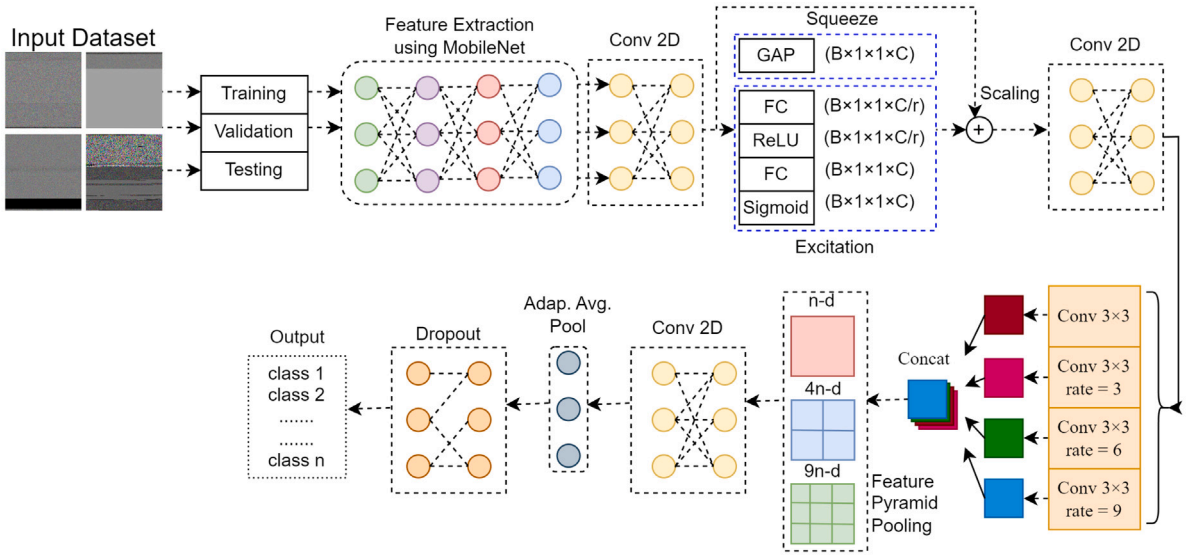

Fig. 1. The architecture of the proposed MSMC-MobileNet for automated malware detection.

and obfuscation. Ultimately, the MSMC-MobileNet model offers a robust method for differentiating between benign and malicious software, especially in IoT environments.

The identification of common patterns within malware classes is a central component for distinguishing different types of malware. The model works by analyzing byteplot images, which visually represent the binary structure of malware files. These images capture key features of malware families, allowing the model to identify distinct patterns linked to specific malicious behaviors. For example, ransomware shows clear byteplot patterns of encrypted file sections, which the model detects through the ASPP module that extracts multi-scale features. On the other hand, Trojans or viruses exhibit irregularities in byte sequences due to behaviors like code injection or data manipulation, which the SE module helps highlight by recalibrating the importance of key features. Furthermore, worms show repetitive structures in their byteplots, which are captured by the FPP module, helping the model identify malware that propagates itself across systems. These visual patterns allow the model to distinguish various malware families based on their typical behaviors and structures.

The model's ability to link specific malware patterns to corresponding behaviors is another critical aspect of its explainability. The byteplot patterns extracted from malware files are not only useful for classification, but also for understanding the malicious behaviors they represent. For example, the presence of irregular or randomized byte sequences often signals obfuscation, a technique used by malware to avoid detection. This behavior is linked to changes in byte structure that differ from benign software, which tends to have structured and predictable byte patterns. Additionally, behaviors such as self-replication, characteristic of worms, are reflected in repetitive byteplot structures that the model identifies through multi-scale feature extraction. Malware behavior like code injection, which involves inserting malicious code into legitimate processes, can be detected through abrupt alterations in byte sequences. The model links these byteplot features directly to the associated malicious activities, thereby providing a robust framework for understanding malware behavior.

The MSMC-MobileNet excels at recognizing the differences in behavior that malware exhibits compared to legitimate applications. Benign software, such as common productivity tools or web browsers, generally exhibits stable, predictable byte sequences, which are consistent in structure and behavior during execution. Malware, in contrast, often shows abnormal behaviors such as encrypted data blocks, frequent system manipulations, or excessive network activity. The model differentiates these by focusing on multi-contextual features and local behaviors observed in the byteplots. For example, benign software would not exhibit the same large file system modifications or network activity spikes that are typical of malicious programs like ransomware. The SE module, along with the ASPP and FPP modules, helps the model to recalibrate important features and capture both local and global patterns, ensuring a clear differentiation between benign software and malware. This ability to detect irregularities in byte sequences and map them to known malicious behaviors allows the model to make highly accurate distinctions between malicious and benign software, providing strong explainability for its decisions.

3.2. MobileNetv3 for feature extraction

MobileNetv3 [14] is an efficient convolutional neural network architecture designed for mobile and embedded vision applications, combining depthwise separable convolutions with lightweight building blocks to achieve a balance between performance and computational cost. It leverages advances from MobileNetV1 and V2, including inverted residuals and linear bottlenecks. It introduces new elements such as squeeze-and-excitation modules and the h-swish activation function to enhance accuracy and efficiency. MobileNetv3 comes in two variants, Large and Small, optimized for different resource constraints and performance needs. The architecture achieves state-of-the-art results on image classification and other vision tasks while maintaining low latency and memory usage, making it suitable for deployment on devices with limited computational power.

3.3. Squeeze and excitation

The Squeeze and Excitation (SE) [17] block is a neural network module designed to improve feature representation by explicitly modeling interdependencies between channels. For malware image classification, the SE block first applies global average pooling to condense the spatial dimensions of each feature map into a single representative value (the "squeeze" step). Then, it uses two fully connected layers with a ReLU and sigmoid activation to generate channel-wise weights (the "excitation" step). These weights are used to scale the original feature maps, emphasizing important features and suppressing less relevant ones. By enhancing the network's focus on significant patterns and structures within the malware images, the SE block can improve the accuracy and robustness of the malware classification model. The architecture of SE is shown in Fig. 2.

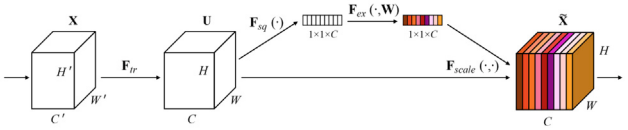

Fig. 2. The standard architecture of SE blocks used in this research [17].

3.3.1. Squeeze

The **squeeze** operation aggregates global spatial information for each channel to generate channel-wise statistics. Given an input feature map $\mathbf{X} \in \mathbb{R}^{H\times W\times C}$, where H, W, and C represent height, width, and the number of channels, respectively, the squeeze operation performs global average pooling using Eq. (1):

$$\mathbf{z}_c = \frac{1}{H\times W}\sum_{i=1}^{H}\sum_{j=1}^{W}\mathbf{X}_{i,j,c}, \quad c = 1,2,\ldots,C. \tag{1}$$

Here, $\mathbf{z} \in \mathbb{R}^C$ is the channel descriptor vector.

3.3.2. Excitation

The **excitation** operation captures channel-wise dependencies and adaptively recalibrates the feature map. A gating mechanism with learnable parameters is applied to the channel descriptor vector $\mathbf{z}$ with Eq. (2):

$$\mathbf{s} = \sigma(\mathbf{W}_2\delta(\mathbf{W}_1\mathbf{z})), \tag{2}$$

Where:

- $\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r}\times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C\times\frac{C}{r}}$ are the weights of two fully connected layers,
- $\delta(\cdot)$ is the ReLU activation function,
- $\sigma(\cdot)$ is the sigmoid activation function,
- r is the reduction ratio that controls the bottleneck in the excitation operation.

The output $\mathbf{s} \in \mathbb{R}^C$ represents the channel-wise recalibration weights.

3.3.3. Recalibration

Finally, the input feature map $\mathbf{X}$ is rescaled by the recalibration weights $\mathbf{s}$ through channel-wise multiplication with Eq. (3):

$$\mathbf{Y}_{i,j,c} = \mathbf{s}_c \cdot \mathbf{X}_{i,j,c}, \quad \forall i,j,c. \tag{3}$$

Here, $\mathbf{Y} \in \mathbb{R}^{H\times W\times C}$ is the recalibrated feature map.

3.4. Atrous Spatial Pyramid Pooling (ASPP)

The ASPP [15] module is a powerful component used in convolutional neural networks to enhance the receptive field of the network by applying multiple parallel atrous (or dilated) convolutions with different rates, thereby enabling the capture of multi-scale contextual information without significantly increasing the number of parameters. By concatenating the feature maps generated by these convolutions, ASPP can effectively aggregate context from various scales, improving the network's ability to understand complex patterns and spatial hierarchies in the input data. This makes ASPP particularly useful for tasks that require detailed spatial understanding, such as segmenting objects in images or distinguishing between different regions within an image. The architecture of the ASPP is shown in Fig. 3.

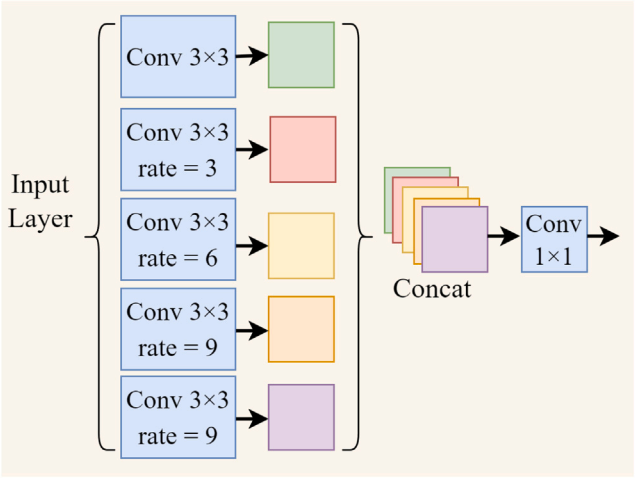

Fig. 3. The standard architecture of the ASPP Module used in this research.

3.4.1. Pruning approach for ASPP

The ASPP (Atrous Spatial Pyramid Pooling) module applies convolutions at multiple dilation rates to capture multi-scale contextual features. However, not all dilation rates contribute equally to the final classification. To prune the ASPP module, we evaluate the cumulative response of each dilation rate d across all channels using Eq. (4):

$$S_{aspp}^{(d)} = \sum_{c=1}^{C} S_c^{(d)} \tag{4}$$

where $S_c^{(d)}$ represents the channel activation at dilation rate d. We retain the top D_{pruned} dilation rates that contribute the most to the total feature extraction, effectively reducing computational complexity without significant performance loss.

Given an input feature map $\mathbf{X} \in \mathbb{R}^{H\times W\times C}$, the ASPP module applies multiple parallel atrous convolutions with different dilation rates $r_1, r_2, \ldots, r_n$. The output of each atrous convolution is computed as Eq. (5):

$$\mathbf{Y}_{i,j,c}^{(k)} = \sum_{m=-K}^{K}\sum_{n=-K}^{K}\mathbf{W}_{m,n,c}^{(k)} \cdot \mathbf{X}_{i+m\cdot r_k, j+n\cdot r_k, c}, \tag{5}$$

where:

- $\mathbf{W}^{(k)}$ is the kernel for the kth atrous convolution,
- K is the kernel size,
- r_k is the dilation rate for the kth convolution.

The module also incorporates global average pooling to capture global context. The pooled feature vector $\mathbf{z}$ is computed as Eq. (6):

$$\mathbf{z}_c = \frac{1}{H\times W}\sum_{i=1}^{H}\sum_{j=1}^{W}\mathbf{X}_{i,j,c}. \tag{6}$$

This vector is broadcast and concatenated with the outputs of the atrous convolutions. The final output of the ASPP module is obtained by concatenating all feature maps and applying a 1×1 convolution for dimensionality reduction using Eq. (7):

$$\mathbf{Y}_{\text{ASPP}} = \text{Conv}_{1\times1}\left(\text{Concat}\left(\mathbf{Y}^{(1)}, \mathbf{Y}^{(2)}, \ldots, \mathbf{Y}^{(n)}, \mathbf{z}\right)\right). \tag{7}$$

3.5. Feature Pyramid Pooling (FPP)

Feature Pyramid Pooling (FPP) [16] is a technique used in convolutional neural networks to enhance feature extraction by aggregating multi-scale contextual information. For malware classification, this approach helps capture various levels of patterns and structures within

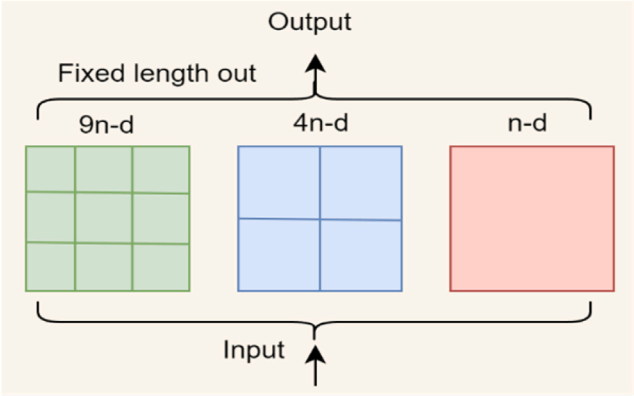

Fig. 4. The standard architecture of the FPP Module used in this research.

malware images, which can be crucial for distinguishing between different types of malware. FPP achieves this by pooling features from different scales and resolutions, then combining them to form a more comprehensive representation. By incorporating features at multiple scales, FPP enables the network to better recognize intricate and hierarchical patterns in malware, thereby enhancing the model's ability to classify diverse and complex malware samples accurately. This multi-scale approach is particularly beneficial in malware analysis, where patterns can vary significantly in size and structure, as shown in Fig. 4.

3.5.1. Pruning approach for FPP

The FPP (Feature Pyramid Pooling) module enhances hierarchical feature extraction by pooling at multiple spatial scales. To optimize its efficiency, we prune pooling scales that provide redundant information. The importance of each pooling scale s is quantified as Eq. (8):

$$S_{fpp}^{(s)} = \sum_{c=1}^{C} S_c^{(s)} \tag{8}$$

where $S_c^{(s)}$ represents the aggregated feature importance at scale s. Similar to ASPP pruning, we retain only the top S_{pruned} pooling scales that significantly contribute to classification accuracy.

Given an input feature map $\mathbf{X} \in \mathbb{R}^{H\times W\times C}$, the FPP module applies to pool operations with varying window sizes. Let P_k denote the kth pooling operation with a window size of s_k. The pooled output for each scale is computed as Eq. (9):

$$\mathbf{Y}_c^{(k)} = \frac{1}{s_k^2} \sum_{i=1}^{s_k} \sum_{j=1}^{s_k} \mathbf{X}_{i,j,c}, \quad c = 1, 2, \ldots, C. \tag{9}$$

After pooling, the feature maps are resized to the same spatial dimensions as the input feature map using bilinear interpolation with Eq. (10):

$$\tilde{\mathbf{Y}}^{(k)} = \text{Interpolate}(\mathbf{Y}^{(k)}, \text{size} = (H, W)). \tag{10}$$

The outputs from all scales are concatenated along the channel dimension with Eq. (11):

$$\mathbf{Y}_{\text{FPP}} = \text{Concat}(\tilde{\mathbf{Y}}^{(1)}, \tilde{\mathbf{Y}}^{(2)}, \ldots, \tilde{\mathbf{Y}}^{(n)}). \tag{11}$$

Finally, a 1×1 convolution is applied to reduce the dimensionality and fuse the features using Eq. (12):

$$\mathbf{Y}_{\text{output}} = \text{Conv}_{1\times1}(\mathbf{Y}_{\text{FPP}}). \tag{12}$$

4. Experimental results and analysis

This section presents the evaluation of the MSMC-MobileNet model on the Malimg and MaleVis datasets using metrics such as accuracy, precision, recall, F1 score, and AUC. The model outperforms existing methods, showing superior detection performance. The use of preprocessing, data augmentation, and dropout regularization improves the model's robustness. Ablation studies confirm that the SE, ASPP, and FPP modules make significant contributions to the model's high efficiency and effectiveness in malware detection.

Table 2
The description of the Malimg and Malevis dataset.

Dataset	No. of images	Classes
Malimg	2339	25
Malevis	14,226	26

Table 3
Details of the Malimg and MaleVis datasets.

Malimg classes	MaleVis classes	Malimg classes	MaleVis classes
Adialer.C	Adposhel	Agent.FYI	Agent
Allaple.A	Allaple	Allaple.L	Amonetize
Alueron.gen!J	Androm	Autorun.K	Autorun
C2LOP.gen!g	BrowseFox	C2LOP.P	Dinwod
Dialplatform.B	Elex	Dontovo.A	Expirio
Fakerean	Fasong	Instantaccess	HackKMS
Lolyda.AA1	Hlux	Lolyda.AA2	Injector
Lolyda.AA3	InstallCore	Lolyda.AT	MultiPlug
Malex.gen!J	Neoreklami	Obfuscator.AD	Neshta
Rbot.gen	Other	Skintrim.N	Regrun
Swizzor.gen!E	Sality	Swizzor.gen!I	Snarasite
VB.AT	Stantinko	Wintrim.BX	VBA
Yuner.A	VBKrypt		Vilsel

4.1. Dataset definition

The Malimg dataset [38] is widely used in cybersecurity research for malware classification. It contains malware samples from 25 different families, with each sample represented as a grayscale image. The dataset was created by converting malware binaries into visual patterns by interpreting the binary content as pixel values. The Malimg dataset has an imbalance issue, where some classes have significantly more samples than others. The experiments are conducted using class weighting to address the imbalance issue present in the Malimg dataset. Class weighting assigns higher importance to underrepresented classes during training, ensuring that the model gives more attention to minority classes. This technique helps mitigate the bias towards majority classes, improving the model's ability to classify all classes, including those with fewer samples, correctly. Sample images of the Malimg and Malevis datasets are shown in Fig. 5.

The MaleVis dataset [39] is also used for malware classification, but it provides a different collection of malware samples, focusing on visualization. Similar to Malimg, it involves transforming malware binaries into image formats, allowing for the use of computer vision techniques to detect and classify malware. Both datasets are valuable for research in malware detection, offering a unique approach that leverages visual patterns in malware binaries to enhance classification accuracy and advance cybersecurity measures.

The Malimg dataset consists of highly imbalanced data with 25 classes, while the MaleVis dataset is a balanced dataset with 26 classes. Both datasets are also combined to form a single dataset that includes all classes. The details of each dataset are given in Table 2, and the details for each class are given in Table 3.

4.2. Evaluation metrics

In machine learning, various evaluation metrics are used to assess the performance of classification models [40]. These metrics are crucial in understanding how well a model performs in predicting both positive and negative instances. Below are the definitions and corresponding equations for the commonly used evaluation metrics:

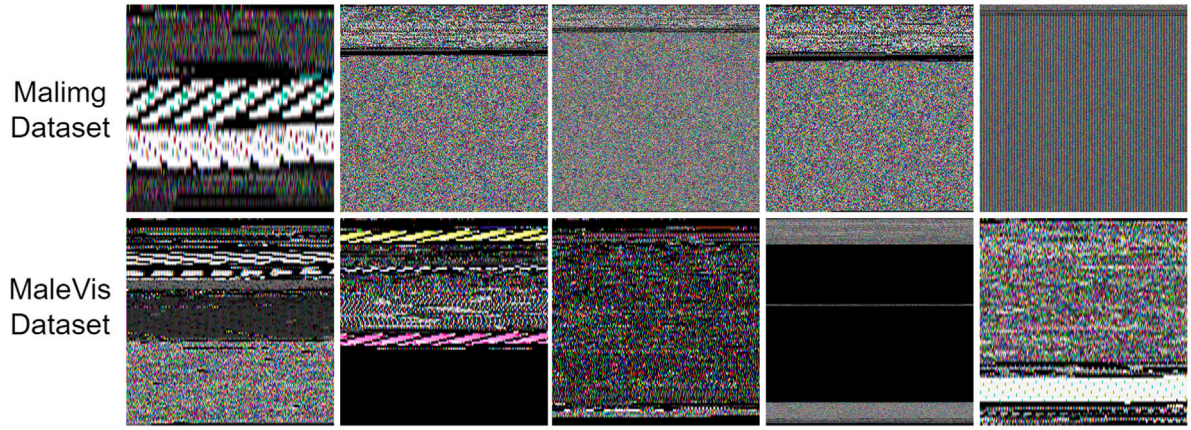

Fig. 5. The sample images of the Malimg and Malevis datasets used in this research.

4.2.1. Accuracy

Accuracy measures the overall correctness of the model by calculating the proportion of correctly classified instances to the total number of instances [40]. It can be calculated as Eq. (13).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{13}$$

where: TP = True Positives (correctly predicted positive cases), TN = True Negatives (correctly predicted negative cases), FP = False Positives (incorrectly predicted as positive), and FN = False Negatives (incorrectly predicted as negative)

4.2.2. Precision

Precision focuses on the performance of the model in predicting positive cases [40]. It is the ratio of correctly predicted positive cases to all predicted positive cases, measuring the accuracy of positive predictions. It can be calculated as Eq. (14).

$$\text{Precision} = \frac{TP}{TP + FP} \tag{14}$$

4.2.3. Recall

Recall, also known as sensitivity or true positive rate, measures the model's ability to identify all relevant positive cases correctly [40]. It is the ratio of correctly predicted positive cases to all actual positive cases. It can be calculated as Eq. (15).

$$\text{Recall} = \frac{TP}{TP + FN} \tag{15}$$

4.2.4. F1 score

The F1 score is the harmonic mean of precision and recall [40]. It provides a single metric that balances both precision and recall, especially useful when dealing with imbalanced datasets. It can be calculated as Eq. (16).

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

4.2.5. AUC (area under the ROC curve)

AUC measures the ability of the model to distinguish between positive and negative classes [40]. It is computed as the area under the Receiver Operating Characteristic (ROC) curve, which is a plot of the True Positive Rate (TPR) versus the False Positive Rate (FPR). The equation for AUC can be expressed as Eq. (17):

$$\text{AUC} = \int_0^1 \text{TPR}(FPR)\, d(FPR) \tag{17}$$

where: TPR = $\frac{TP}{TP+FN}$ (True Positive Rate), and FPR = $\frac{FP}{FP+TN}$ (False Positive Rate)

A higher AUC value indicates better model performance in distinguishing between the two classes across various thresholds.

4.3. Model training parameters

For the training of the proposed MSMC-MobileNet model, the dataset is divided into three parts: 70% of the dataset is used for training, 10% of the dataset is used for validation, and 20% of the dataset is used for testing. The initial learning rate is 0.001, and there are 150 training epochs. The batch size is 32, with Adam as the optimizer and cross-entropy as the loss function. The early stopping method is also used to terminate the training process if there is no improvement in the training after five epochs. First, the experiments are performed individually on both datasets. After that, the datasets are combined to form a large dataset with more input images and classes. The training loss and training accuracy are shown in Fig. 6.

The training loss and accuracy curves indicate that the model improves steadily over time, with training accuracy reaching nearly 1.0 and the loss decreasing as the epochs progress. Initially, the loss decreases sharply, but noticeable fluctuations occur between epochs 20 and 80, indicating some instability or difficulty in learning during this phase. Despite these oscillations, the accuracy curve shows a rapid increase. While the high accuracy and decreasing loss suggest effective learning, the oscillations indicate potential overfitting or instability, particularly during the middle epochs. These fluctuations suggest the model could benefit from further fine-tuning, such as adjusting the learning rate or adding regularization techniques, to achieve smoother convergence and prevent overfitting.

4.4. Results

In this section, experiments are conducted on the proposed MSMC-MobileNet model to evaluate its effectiveness. Different techniques are applied to increase the efficiency and accuracy of the proposed model. By examining byteplot images, the model can detect malicious behaviors in malware by recognizing patterns like unusual byte sequences and alterations in file structures. These anomalies often indicate characteristics such as obfuscation, code injection, or data exfiltration, which are common in malicious software. The model captures these subtle variations in the binary content of executable files, allowing it to distinguish between benign and harmful software based on how the bytes are organized and structured within the file.

4.4.1. Results with preprocessing

To feed the dataset into the model, the preprocessing method is applied, in which all the images are resized to 224 × 224 dimensions. The size of all images differs for each class, and the dataset must be in the same dimension before being fed into the model. The results of the proposed model are given in Table 4. The results of the proposed model surpass those of state-of-the-art methods.

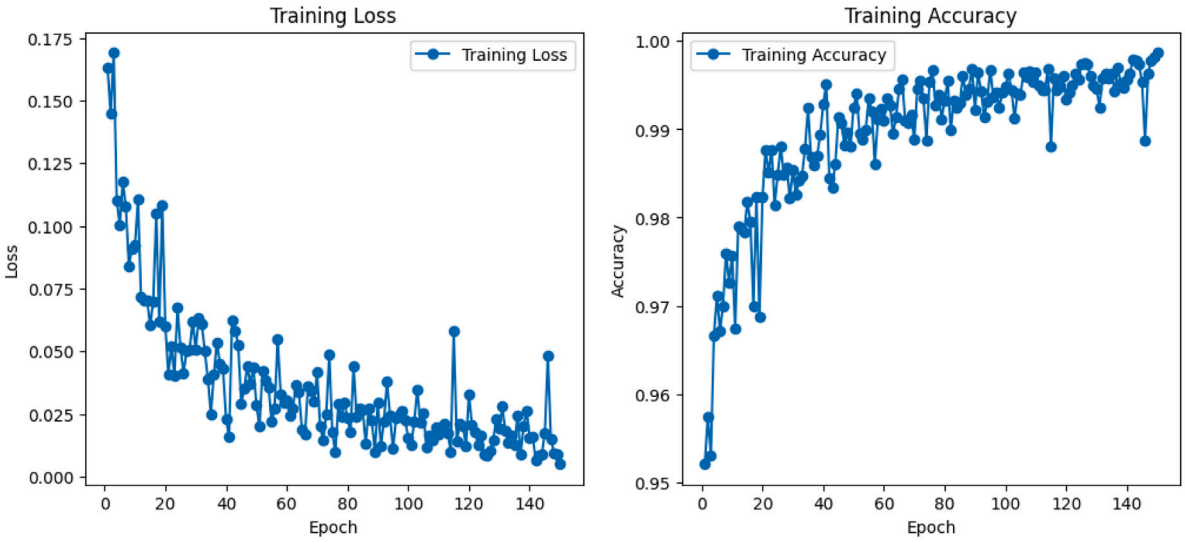

Fig. 6. The training loss and accuracy diagram of the proposed model.

Table 4
The results of the proposed MSMC-MobileNet for malware classification with preprocessing.

Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	AUC (%)
Malimg	83.17	87.05	86.79	87.12	0.95
MaleVis	84.56	87.91	87.43	88.01	0.97
Malimg + MaleVis	87.32	90.64	89.84	90.57	0.99

Table 5
The results of the proposed MSMC-MobileNet for malware classification with preprocessing and data augmentation.

Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	AUC (%)
Malimg	88.36	92.17	93.08	92.83	0.91
MaleVis	90.47	95.02	94.72	93.56	0.94
Malimg + MaleVis	93.82	97.51	96.93	95.84	0.99

For the Malimg dataset, the model achieved an accuracy of 83.17%, precision of 87.05%, recall of 86.79%, F1 score of 87.12%, and AUC of 95.61%. In the case of the MaleVis dataset, the model's accuracy was 84.56%, precision was 87.91%, recall was 87.43%, F1 score was 88.01%, and AUC was 97.58%. When combining the datasets, the model performed the best, with an accuracy of 87.32%, precision of 90.64%, recall of 89.84%, F1 score of 90.57%, and a high AUC of 99.84%.

4.4.2. Results with data augmentation

Data augmentation for malware detection involves creating additional synthetic data to enhance the training of deep-learning models. This technique is crucial due to the limited availability and diversity of malware samples compared to benign software. It helps in building robust models that generally serve unseen threats better. This research utilizes scaling, rotation, and flipping to augment the dataset. Table 5 shows the results of the proposed model after applying data augmentation.

For the Malimg dataset, the model achieved an accuracy of 88.36%, precision of 92.17%, recall of 93.08%, F1 score of 92.83%, and AUC of 91.96%. On the MaleVis dataset, the model performed even better with an accuracy of 90.47%, precision of 95.02%, recall of 94.72%, F1 score of 93.56%, and AUC of 94.63%. The combined dataset yielded the highest results, with an accuracy of 93.82%, precision of 97.51%, recall of 96.93%, F1 score of 95.84%, and an AUC of 99.91%, indicating a significant improvement due to the inclusion of data augmentation.

4.4.3. Results with dropout regularization

Dropout regularization [41] for malware detection is a technique used to prevent overfitting in machine learning models by randomly deactivating a fraction of neurons during each training iteration. This approach ensures that the model does not rely too heavily on any particular set of neurons, promoting the development of a more generalized and resilient model. In the context of malware detection, dropout helps the model distinguish between benign and malicious software more effectively by forcing it to learn diverse and robust features from the input data. Consequently, dropout regularization enhances the model's ability to detect various types of malware, including previously unseen variants, thereby improving overall detection accuracy and reliability. The results of the proposed approach, after applying dropout regularization, are presented in Table 6.

For the Malimg dataset, the model achieved an accuracy of 92.37%, precision of 96.54%, recall of 95.84%, F1 score of 95.47%, and AUC of 98.59%. On the MaleVis dataset, the model achieved an accuracy of 95.08%, precision of 98.33%, recall of 97.90%, F1 score of 98.15%, and AUC of 96.98%. The combined dataset achieved the highest performance, with an accuracy of 98.79%, precision of 99.84%, recall of 99.73%, F1 score of 99.89%, and AUC of 1.00%, indicating a substantial improvement with the application of dropout regularization.

The patterns observed in byteplot images of malware are closely linked to the specific behaviors demonstrated by the target malware. For example, ransomware exhibits distinctive byteplot patterns representing encrypted file sections, reflecting its behavior of file encryption for ransom demands. Trojans and viruses, which often involve code

Table 6
The results of the proposed MSMC-MobileNet for malware classification with preprocessing, data augmentation, and dropout regularization.

Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	AUC (%)
Malimg	92.37	96.54	95.84	95.47	0.98
MaleVis	95.08	98.33	97.90	98.15	0.96
Malimg + MaleVis	98.79	99.84	99.73	99.89	1

injection or data manipulation, display irregular byte sequences, indicative of alterations to the binary structure. Worms, known for their self-replicating behavior, show repetitive byte structures in their byteplots, corresponding to their propagation across systems. These characteristic patterns, linked to the malware's actions such as encryption, code injection, and self-replication, are effectively captured by the MSMC-MobileNet model using its multi-scale and multi-contextual feature extraction modules like ASPP, FPP, and SE. These modules enable the model to focus on essential features, such as hierarchical and multi-scale details, to link the observed byteplot patterns to the underlying malware behaviors. This approach not only improves the model's accuracy in differentiating between benign and malicious software but also enhances its ability to detect and classify new and previously unseen variants of the same malware family, offering a robust solution for real-time malware detection in resource-constrained environments.

The byteplot images of malware exhibit distinguishable patterns and characteristics compared to benign software, enabling effective detection. Malware byteplots often display irregularities such as encrypted file sections, abnormal byte sequences, or repetitive structures, which correspond to behaviors like encryption, code injection, or self-replication. In contrast, benign software typically shows stable, predictable byte sequences with minimal structural variance. These distinct visual differences between malware and benign software are crucial for distinguishing between the two, allowing the MSMC-MobileNet model to accurately identify and classify malware based on its unique byteplot patterns, while recognizing benign software by its consistent and predictable byte structure.

4.5. K-fold cross validation

Table 7 presents the results of the 5-fold cross-validation for the MSMC-MobileNet model, which is used for malware detection and classification. The table includes performance metrics such as Accuracy, Precision, Recall, F1 Score, and AUC for each fold, as well as their averages across the five folds. The Accuracy metric ranges from 95.12% to 95.72% for individual folds, with an average of 95.34%. This shows that the model consistently performed well in correctly classifying both malware and benign instances across all folds. Precision values range from 97.58% to 98.15%, indicating that the model is highly accurate in predicting positive (malware) cases. Similarly, Recall values show a consistent performance from 98.18% to 98.50%, indicating the model's strong ability to identify all relevant positive instances.

In terms of F1 Score, which balances Precision and Recall, the model achieved values ranging from 97.85% to 98.06%, with an average of 97.95%, further highlighting its robustness in malware detection. The AUC values were consistently high, ranging from 98.99% to 99.15%, with an average of 99.07%. The high AUC indicates the model's excellent ability to distinguish between positive (malware) and negative (benign) classes across different thresholds. These results demonstrate that the MSMC-MobileNet model performs effectively in terms of both detection accuracy and computational efficiency, with excellent performance across all evaluation metrics.

4.6. Ablation study

To obtain the best results from the proposed MSMC-MobileNet model, ablation experiments are performed. Different experiments are performed to determine the efficiency of the proposed model. The results of the ablation experiments are given in Table 8. The addition of multiple modules enhances the efficiency and accuracy of the proposed model. The confusion matrix of the proposed model for all the given classes is shown in Fig. 7.

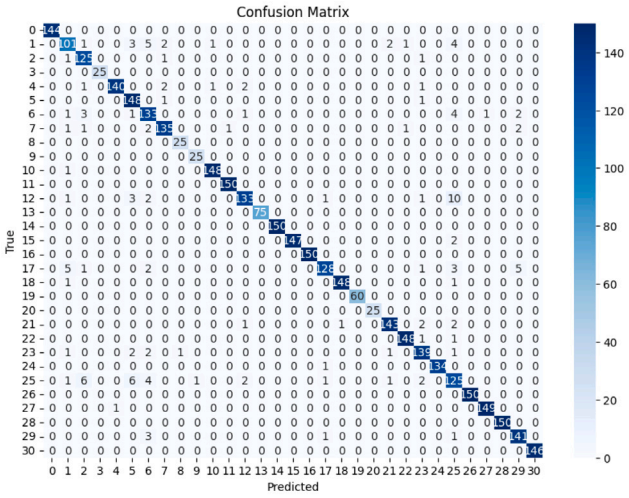

Fig. 7. The confusion matrix of malware detection for the proposed model on the ablation study.

4.6.1. Effectiveness of MobileNetv3

First, MobileNetv3 is used for malware classification. The Malimg dataset achieves accuracy, precision, recall, F1 score, and AUC as 85.61%, 87.29%, 86.94%, 87.62%, and 92.53%, respectively. The accuracy, precision, recall, F1 score, and AUC of the MaleVis dataset are 82.36%, 84.75%, 85.61%, 85.18%, and 94.53%, respectively. The results of MobileNetv3, combined with the Malimg and MaleVis datasets, for accuracy, precision, recall, F1 score, and AUC are 87.95%, 90.52%, 89.43%, 90.34%, and 90.42%, respectively.

4.6.2. Effectiveness of MobileNetv3 with SE blocks

The standard architecture of the MobileNetv3 is modified by adding SE blocks. The results of this model are better than the standard MobileNetv3 model for all three datasets. The MSMC-MobileNet achieves accuracy, precision, recall, F1 score, and AUC as 88.37%, 91.47%, 90.91%, 92.03%, and 91.75% for the Malimg dataset while the MaleVis dataset achieves 88.24%, 91.58%, 91.07%, 92.14%, and 91.68% respectively. The combined dataset achieves 90.65%, 94.35%, 93.98%, 94.62%, and 94.51% of accuracy, precision, recall, F1 score, and AUC.

4.6.3. Effectiveness of MobileNetv3 with SE and ASPP blocks

By incorporating the ASPP and SE modules into MobileNetv3, the accuracy, precision, recall, F1 score, and AUC for the Malimg dataset are 90.61%, 93.64%, 94.11%, 93.87%, and 94.52%, respectively. The MaleVis dataset achieves 93.53%, 96.95%, 95.48%, 95.89%, and 96.04% as accuracy, precision, recall, F1 score, and AUC. The accuracy, precision, recall, F1 score, and AUC for the combined dataset are 95.42%, 98.92%, 97.83%, 98.51%, and 97.22%, respectively. The standard architecture of MobileNetv3 with ASPP and SE achieves better results compared to previous models.

Table 7
Results of 5-fold cross-validation for MSMC-MobileNet model.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	AUC (%)
Fold 1	95.12	97.58	98.22	97.89	98.99
Fold 2	95.56	97.85	98.30	98.07	99.12
Fold 3	95.23	97.92	98.18	97.85	99.03
Fold 4	95.45	98.03	98.38	97.91	99.08
Fold 5	95.72	98.15	98.50	98.06	99.15
Average	**95.34**	**97.91**	**98.12**	**97.95**	**99.07**

Table 8
The results of the MSMC-MobileNet for malware detection with ablation experiments.

Model	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)	AUC (%)
MobileNetv3	Malimg	85.61	87.29	86.94	87.62	0.92
	MaleVis	82.36	84.75	85.61	85.18	0.84
	Malimg + MaleVis	87.95	90.52	89.43	90.34	0.90
MobileNetv3 + SE	Malimg	88.37	91.47	90.91	92.03	0.91
	MaleVis	88.24	91.58	91.07	92.14	0.91
	Malimg + MaleVis	90.65	94.35	93.98	94.62	0.94
MobileNetv3 + SE + ASPP	Malimg	90.61	93.64	94.11	93.87	0.94
	MaleVis	93.53	96.95	95.48	95.89	0.96
	Malimg + MaleVis	95.42	98.92	97.83	98.51	0.97
Proposed MSMC-MobileNet	Malimg	92.37	96.54	95.84	95.47	0.98
	MaleVis	95.08	98.33	97.90	98.15	0.96
	Malimg + MaleVis	98.79	99.84	99.73	99.89	1

4.6.4. Proposed MSMC-MobileNet

The proposed model is a modified version of MobileNetV3, which incorporates ASPP, SE, and FPP for multi-scale and multi-contextual feature extraction. The results of the proposed model for the Malimg dataset are 92.37%, 96.54%, 95.84%, 95.47%, and 98.59% for accuracy, precision, recall, F1 score, and AUC, respectively. In contrast, the MaleVis dataset achieves 95.08%, 98.33%, 97.9%, 98.15%, and 96.98% as accuracy, precision, recall, F1 score, and AUC, respectively. The accuracy, precision, recall, F1 score, and AUC for the combined dataset are 98.79%, 99.84%, 99.73%, 99.89%, and 100, respectively. The results of the proposed model outperform those of previous methods for malware detection and classification.

The ablation study reveals that each component of the MSMC-MobileNet model contributes to its enhanced performance in malware detection. Starting with the baseline MobileNetv3, the addition of SE blocks improves the model's results across all datasets, boosting accuracy and other metrics. Further improvements are observed with the inclusion of ASPP and SE blocks, leading to better feature extraction and higher performance on both individual datasets and their combination. Finally, the proposed MSMC-MobileNet, which integrates ASPP, SE, and FPP modules, achieves the highest accuracy, precision, recall, F1 score, and AUC, surpassing all previous models. The results on the combined Malimg and MaleVis datasets demonstrate a significant leap, confirming the effectiveness of the proposed model in malware classification tasks.

4.7. Comparative analysis of the proposed model

This section presents a comprehensive comparison between the proposed MSMC-MobileNet model and several existing deep learning-based approaches for malware detection. The comparison results, as shown in Table 9, highlight the performance of different models in terms of key evaluation metrics, including accuracy, precision, recall, and F1 score.

Jian et al. [23] proposed the SERLA model, which integrates SEResNet50, Bi-LSTM, and an Attention module for malware classification. The SERLA model achieved a respectable performance, with an accuracy of 98.31%, precision of 98.68%, recall of 97.93%, and an F1 score of 98.30%. In a similar vein, Islam et al. [24] introduced the Jadeite model, which leverages byte images for malware detection. The Jadeite model's performance, with an accuracy of 94.8% and an F1 score of 95%, reflects a strong but lower result compared to the proposed model. Furqan et al. [25] employed a CNN-based approach, which demonstrated solid performance with an accuracy of 97%, precision of 96%, recall of 96%, and F1 score of 96%. While robust, this model falls short in comparison to the MSMC-MobileNet's capabilities. Mohammad et al. [29] focused on a CNN model that utilizes executable binaries represented as grayscale images in the frequency domain via the Discrete Cosine Transform (DCT). Despite achieving an accuracy of 83.60%, this approach showed limited performance compared to more advanced models. Jamal et al. [30] also used a CNN and DNN-based approach for malware classification, achieving an accuracy of 93.5%, precision of 93.4%, and F1 score of 94.1%. While it highlights the challenges of malware complexity and volume, it still lags behind the proposed model's results.

In contrast, the proposed MSMC-MobileNet model, a modified version of MobileNetv3, incorporates advanced modules such as SE, ASPP, and FPP to enhance feature extraction across multiple scales and contexts. The SE module enriches the extracted features, enabling the model to capture complex patterns in the data. Extensive experiments conducted on the Malimg and MaleVis datasets demonstrate the effectiveness of the MSMC-MobileNet model. The model achieves an impressive accuracy of 92.37%, precision of 96.54%, recall of 95.84%, F1 score of 95.47%, and AUC of 98.59% on the Malimg dataset. On the MaleVis dataset, the model performs even better, with an accuracy of 95.08%, precision of 98.33%, recall of 97.9%, F1 score of 98.15%, and AUC of 96.98%. When combined, the model reaches an outstanding accuracy.

5. Conclusion

In this research, a novel MSMC-MobileNet model is proposed for the detection and classification of malware using byteplot images. The Malimg and MaleVis datasets are used for evaluating the proposed model. First, MobileNetv3 is used for malware classification. After that, the SE module is added to MobileNetv3, and the results are improved due to the extraction of important information. To extract multi-scale and multi-contextual information from the input dataset, ASPP and FPP modules are also added. The data augmentation methods, such as scaling, rotation, flipping, and translation, are also used to increase the number of input dataset images. Dropout regularization is also used

Table 9
The comparison of the proposed model with other deep learning models for malware detection.

Refs.	Method	Dataset	Accuracy (%)	Precision (%)	Recall (%)	F1 score (%)
[23]	SERLA	Byte files	98.31	98.68	97.93	98.30
[24]	Jadeite	Byte files	94.8	95	95	95
[25]	CNN	Byte files	97	96	96	96
[29]	CNN	Byte files	83.60	–	–	–
[30]	CNN	Byte files	93.5	93.4	95	94.1
Proposed	MSMC-MobileNet	Byte files	98.79	99.84	99.73	99.89

to reduce overfitting problems. From the results, the MSMC-MobileNet outperforms state-of-the-art methods.

The limitations of this study include potential biases in the dataset and the lack of consideration for real-world deployment challenges, such as longer running times on resource-constrained devices. Future work could focus on expanding the dataset to improve generalization, enhancing the model's scalability, and exploring advanced regularization techniques to mitigate overfitting.

CRediT authorship contribution statement

Sidra Javed: Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation. **Guowei Wu:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Data curation, Conceptualization. **Hamza Javed:** Visualization, Validation, Investigation, Formal analysis, Data curation, Conceptualization. **Osama A. Khashan:** Writing – review & editing, Supervision, Resources, Project administration, Methodology. **Haseeb Hassan:** Writing – original draft, Software, Methodology, Formal analysis, Conceptualization. **Anwar Ghani:** Writing – review & editing, Writing – original draft, Visualization, Supervision, Project administration, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research work was supported by the Social Policy Grant (SPG) of the Nazarbayev University, Kazakhstan.

Data availability

The datasets used in this research are available publicly and can be found at the following links:

- **Malimg:** https://www.kaggle.com/datasets/manmandes/malimg.
- **MaleVis:** https://web.cs.hacettepe.edu.tr/~selman/malevis/.

References

[1] Khan KN, Ullah N, Ali S, Khan MS, Nauman M, Ghani A. Op2Vec: An opcode embedding technique and dataset design for end-to-end detection of android malware. Secur Commun Networks 2022;2022(1):3710968. http://dx.doi.org/10.1155/2022/3710968.

[2] Kolar D, Challagidad P. A review on identification and classification of malware attacks. J Emerg Technol Innov Res 2021;8.

[3] Ojugo A, Eboka A. Signature-based malware detection using approximate Boyer Moore string matching algorithm. Int J Math Sci Comput 2019;5(3):49–62.

[4] Javed A, Akhlaq M. On the approach of static feature extraction in trojans to combat against zero-day threats. In: 2014 international conference on IT convergence and security. IEEE; 2014, p. 1–5.

[5] Nirmala MS. Malware image detection by using convolutional neural network. 11, 2023, URL https://api.semanticscholar.org/CorpusID:262881501,

[6] Vinayakumar R, Alazab M, Soman K, Poornachandran P, Venkatraman S. Robust intelligent malware detection using deep learning. IEEE Access 2019;7:46717–38.

[7] Shaik A, Pendharkar G, Kumar S, Balaji S, et al. Comparative analysis of imbalanced malware byteplot image classification using transfer learning. 2023, arXiv preprint arXiv:2310.02742.

[8] Deepa K, Adithyakumar K, Vinod P. Malware image classification using vgg16. In: 2022 international conference on computing, communication, security and intelligent systems. IEEE; 2022, p. 1–6.

[9] Khan AR, Yasin A, Usman SM, Hussain S, Khalid S, Ullah SS. Exploring lightweight deep learning solution for malware detection in IoT constraint environment. Electronics 2022;11(24):4147.

[10] Liu Y, Bai X, Liu Q, Lan T, Zhou L, Zhou T. Meta-HFMD: A hierarchical feature fusion malware detection framework via multi-task meta-learning. In: International conference on frontiers in cyber security. Springer; 2023, p. 638–54.

[11] Naeem H, Batool A. Malware attacks detection in network security using deep learning approaches. Int J Electron Crime Investig 2023;7(3):35–48.

[12] AlGarni MD, AlRoobaea R, Almotiri J, Ullah SS, Hussain S, Umar F. An efficient convolutional neural network with transfer learning for malware classification. Wirel Commun Mob Comput 2022;2022(1):4841741.

[13] Alamro H, Mtouaa W, Aljameel S, Salama AS, Hamza MA, Othman AY. Automated android malware detection using optimal ensemble learning approach for cybersecurity. IEEE Access 2023.

[14] Howard A, Sandler M, Chu G, Chen L-C, Chen B, Tan M, Wang W, Zhu Y, Pang R, Vasudevan V, et al. Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision. 2019, p. 1314–24.

[15] Chen L-C, Papandreou G, Kokkinos I, Murphy K, Yuille AL. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. IEEE Trans Pattern Anal Mach Intell 2017;40(4):834–48.

[16] Khan SR, Raza A, Waqas M, Zia MAR. Efficient and accurate image classification via spatial pyramid matching and SURF sparse coding. Lahore Garrison Univ Res J Comput Sci Inf Technol 2023;7(4).

[17] Hu J, Shen L, Sun G. Squeeze-and-excitation networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2018, p. 7132–41.

[18] Habib H, Rafique I. A systematic literature review on malware attacks. J Comput Artif Intell 2023. URL https://api.semanticscholar.org/CorpusID:270048525.

[19] Javed H, Zeng F, Shaheer M. A novel deep ensemble framework for IoT malware variant detection. In: 2024 IEEE international conference on smart internet of things. 2024, p. 457–62, URL https://api.semanticscholar.org/CorpusID:274827650.

[20] Arse M, Sharma K, Bindewari S, Tomar A, Patil H, Jha N. Mitigating malware attacks using machine learning: A review. In: 2023 international conference on artificial intelligence and smart communication. IEEE; 2023, p. 1032–8.

[21] Nafiiev A, Rodionov A. Malware detection system based on static and dynamic analysis and using machine learning. Theor Appl Cybersecur 2023;5(2).

[22] Aziz S, Irshad M, Haider SA, Wu J, Deng DN, Ahmad S. Protection of a smart grid with the detection of cyber-malware attacks using efficient and novel machine learning models. Front Energy Res 2022;10:964305.

[23] Jian Y, Kuang H, Ren C, Ma Z, Wang H. A novel framework for image-based malware detection with a deep neural network. Comput Secur 2021;109:102400.

[24] Obaidat I, Sridhar M, Pham KM, Phung PH. Jadeite: a novel image-behavior-based approach for java malware detection using deep learning. Comput Secur 2022;113:102547.

[25] Rustam F, Ashraf I, Jurcut AD, Bashir AK, Zikria YB. Malware detection using image representation of malware data and transfer learning. J Parallel Distrib Comput 2023;172:32–50.

[26] Nazim S, Alam MM, Rizvi SA, Mustapha JC, Hussain SS, Su'ud MM. Multimodal malware classification using proposed ensemble deep neural network framework. Sci Rep 2025;15. URL https://api.semanticscholar.org/CorpusID:278859006.

[27] Alam I, Samiullah M, Asaduzzaman SM, Kabir U, Aahad AM, Woo SS. MIRACLE: Malware image recognition and classification by layered extraction. Data Min Knowl Discov 2024;39:10, URL https://api.semanticscholar.org/CorpusID:274871576.

[28] Anand S, Mitra B, Dey S, Rao A, Dhar R, Vaidya J. MALITE: Lightweight malware detection and classification for constrained devices. 2024, arXiv, abs/2309.03294, URL https://api.semanticscholar.org/CorpusID:261582382.

[29] Mohammed TM, Nataraj L, Chikkagoudar S, Chandrasekaran S, Manjunath B. Malware detection using frequency domain-based image visualization and deep learning. 2021, arXiv preprint arXiv:2101.10578.

[30] El Abdelkhalki J, Ahmed MB, Abdelhakim BA. Image malware detection using deep learning. Int J Commun Networks Inf Secur (IJCNIS) 2022;12(2).
[31] Farfoura ME, Mashal I, Alkhatib A, Batyha RM, Rosiyadi D. A novel lightweight machine learning framework for IoT malware classification based on matrix block mean downsampling. Ain Shams Eng J 2025. URL https://api.semanticscholar.org/CorpusID:274751981.
[32] Dong S, Shu L, Nie S. Android malware detection method based on CNN and DNN bybrid mechanism. IEEE Trans Ind Informatics 2024.
[33] Guo K, Xin Y, Yu T. Malware detection using contrastive learning based on multi-feature fusion. In: 2023 IEEE 22nd international conference on trust, security and privacy in computing and communications. 2023, p. 1681–6, URL https://api.semanticscholar.org/CorpusID:270097565.
[34] Yang X, Yang D, Li Y. A hybrid attention network for malware detection based on multi-feature aligned and fusion. Electronics 2023;12(3):713.
[35] Dai Y, Li H, Qian Y, Yang R, Zheng M. SMASH: A malware detection method based on multi-feature ensemble learning. IEEE Access 2019;7:112588–97.
[36] Ramesh S, Selvarayan S, Raghav VM, Arumugam C. A multistage malware detection and classification model using visual features. In: 2023 4th international conference on communication, computing and industry 6.0. IEEE; 2023, p. 1–6.
[37] Hashemi H, Samie ME, Hamzeh A. IFMD: image fusion for malware detection. J Comput Virol Hacking Tech 2023;19(2):271–86.
[38] Reilly C, O. Shaughnessy S, Thorpe C. Robustness of image-based malware classification models trained with generative adversarial networks. In: Proceedings of the 2023 European interdisciplinary cybersecurity conference. 2023, p. 92–9.
[39] MaleVis Dataset. MaleVis: A dataset for vision based malware recognition. 2024, https://web.cs.hacettepe.edu.tr/~selman/malevis/. [Accessed: 09 October 2024].
[40] Idrees AM, Alsheref FK. A collaborative evaluation metrics approach for classification algorithms. J Southwest Jiaotong Univ 2020. URL https://api.semanticscholar.org/CorpusID:218999448.
[41] Salehin I, Kang D-K. A review on dropout regularization approaches for deep neural networks within the scholarly domain. Electronics 2023;12(14):3106.